Università degli Studi di Salerno

Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica Applicata

Corso di Laurea Magistrale in Ingegneria Informatica

Sistemi Embedded
Canale I-Z

Project Work:
Progettare e realizzare il firmware per il sistema di controllo di un cancello elettrico a battente.

Gruppo 6

Cognome Nome	Matricola	E-mail
Lettieri Alessia	0622702324	a.lettieri36@studenti.unisa.it
Napolitano Domenico	0622702325	d.napolitano18@studenti.unisa.it
Panico Marco	0622702416	m.panico20@studenti.unisa.it
Passanante Alessandro	0622702333	a.passannante1@studenti.unisa.it

Anno accademico 2023-2024

Sommario

Progettazione del sistema *3*

1.1 User Stories **3**

1.2 Use Case **5**

Accensione del dispositivo 5
Configurazione delle impostazioni del cancello 5
Funzionamento 6
Presenza ostacoli 7
Verifica chiusura 8

1.3 Activity Diagrams **10**

Apertura/Chiusura 10
Rilevamento ostacoli 11
Configurazione dei tempi di operazione del cancello 12
Accensione del sistema 13
Verifica corretta chiusura del cancello 14

1.4 State diagram **15**

Modellazione del sistema nell'ambiente Simulink *17*

2.1 Descrizione del modello Stateflow **17**

Control System Cancello 17
Button chart 18
Lampeggio led chart 19
Cancello chart 21

Validazione *27*

3.1 Test Led **27**

test_led_on 27
test_led_enable 27

3.2 Test Funzionamento Cancello **28**

apertura_chiusura 28
apertura_chiusura 28
Condizione_errore 28

3.3 Test Cancello Ostacoli **28**

ostacolo_cancello_chiuso 28
ostacolo_cancello_in_movimento 29

Codice sorgente e dimostratore hardware *30*

4.1 Realizzazione del circuito **30**

4.2 Generazione del codice sorgente **32**

Indice delle Figure *33*

Progettazione del sistema

Questa sezione dettaglia la metodologia di progettazione adottata e presenta diversi diagrammi UML (Unified Modelling Language) per illustrare l'architettura e il funzionamento del sistema.

Nota: I diagrammi UML inseriti nel documento sono presenti in formato pdf per una maggiore leggibilità nella cartella Diagrammi.

1.1 User Stories

Si riportano in seguito i requisiti in formato user story per la descrizione delle funzionalità che il sistema di controllo di un cancello elettrico a battente deve possedere.

1. Un utente, a cancello chiuso o in chiusura, può con la pressione del *pulsante di apertura* attivare la fase di apertura del cancello. Il *led di movimento* lampeggia con frequenza 0,5 Hz.

2. Un utente, a cancello aperto o in apertura, può con la pressione del *pulsante di apertura* attivare la fase di chiusura del cancello. Il *led di movimento* lampeggia con frequenza 0,5 Hz.

3. Un utente, a cancello chiuso, può decidere dopo quanto tempo dall'apertura il cancello deve richiudersi attraverso il *pulsante per la chiusura automatica*. Ad ogni pressione corrisponde un incremento di 10 secondi, una volta raggiunto il valore massimo di 120 secondi ritorna a 10.

4. Un utente, a cancello chiuso, può decidere la durata delle fasi di apertura e di chiusura del cancello tramite il *pulsante del tempo di lavoro*. Ad ogni pressione corrisponde un incremento di 10 secondi, una volta raggiunto il valore massimo di 120 secondi ritorna a 10.

5. Un utente vuole che il cancello si riapra automaticamente quando il *sensore di presenza ostacoli* rileva un ostacolo durante la chiusura in modo da evitare collisioni o danni.

6. Un utente vuole che la pressione del *pulsante di apertura*, a cancello chiuso o aperto, sia ignorato quando il sensore di presenza ostacoli è attivo. Il *led di presenza ostacoli* lampeggia per 30 secondi con frequenza di 1Hz.

7. Un utente vuole che il dispositivo entri in uno stato di errore quando il *sensore di chiusura* non si attiva entro la fine del tempo di lavoro.

8. Un utente, dopo 10 secondi di permanenza del dispositivo in stato di errore, vede il *led di errore* acceso.

9. Un utente, quando il cancello è aperto, vede *tutti i led* accesi.

10. Un utente, quando il cancello è chiuso, vede *tutti i led* spenti.

11. Un utente vuole che il dispositivo avvii la procedura di chiusura del cancello quando viene acceso e il *sensore di chiusura* non risulta attivo in modo da garantire che il cancello sia chiuso correttamente, in accordo con il sensore di presenza ostacoli.

12. Un utente vuole che il cancello sia considerato chiuso quando il *sensore di chiusura* è attivo in modo da avere una conferma dello stato del cancello.

1.2 Use Case

L'utente è l'attore principale che interagisce con il sistema di controllo del cancello automatico. Il diagramma dei casi d'uso rappresenta le varie funzionalità e scenari operativi del cancello, descrivendo come l'utente può controllare e monitorare il cancello attraverso diverse azioni e come il sistema risponde a queste azioni. Di seguito sono elencati i casi d'uso derivati da ciascuna storia degli utenti sviluppati in base alle storie degli utenti menzionate in precedenza.

Accensione del dispositivo

Il dispositivo viene acceso e il sistema verifica lo stato del sensore di chiusura.

- Se il sensore di chiusura è attivo, il dispositivo è pronto per l'uso.
- Se il sensore di chiusura non è attivo, viene avviata automaticamente la procedura di chiusura del cancello

User stories corrispondente 11

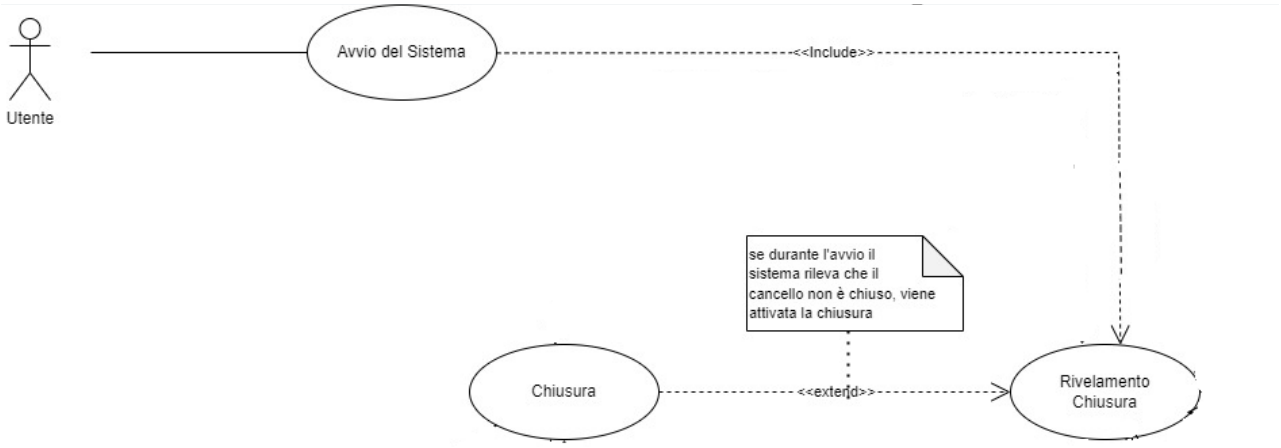

Figura 1: Accensione del dispositivo

Configurazione delle impostazioni del cancello

L'utente configura:

- il tempo per la chiusura automatica del cancello.
- Il tempo di lavoro, cioè la durata delle fasi di apertura e chiusura del cancello

Nota: Entrambe le configurazioni prevedono un valore compreso tra 10 e 120 secondi con un incremento di 10 ad ogni interazione con l'utente.

Nota: Per la configurazione delle impostazioni è necessario che il cancello sia chiuso

User stories corrispondenti 3 e 4

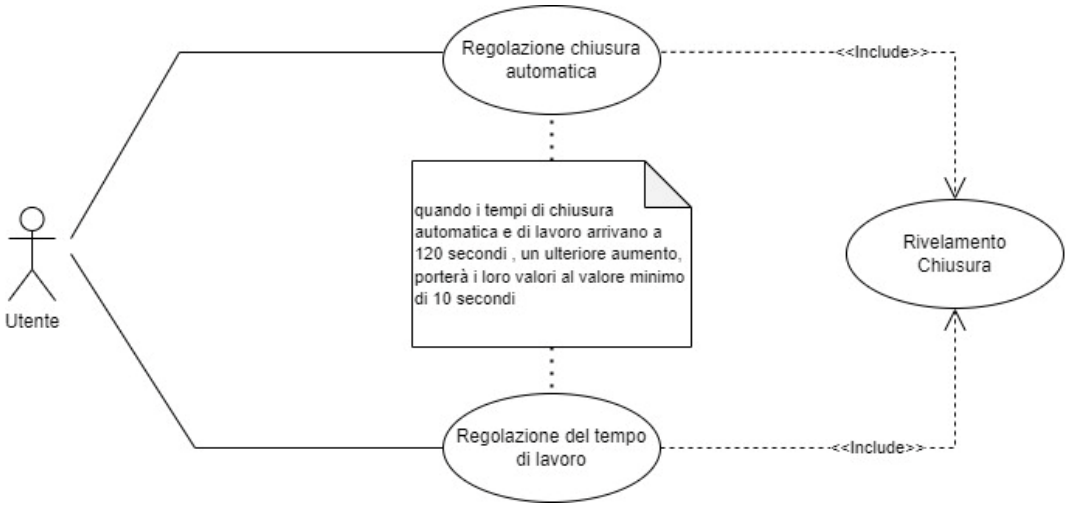

Figura 2: Configurazione delle impostazioni del cancello

Funzionamento

Durante il normale funzionamento, l'utente può:

- aprire il cancello premendo il pulsante di apertura quando il cancello è chiuso o in fase di chiusura.
- chiudere il cancello premendo lo stesso pulsante quando il cancello è aperto o in fase di apertura.

> Nota: Il pulsante è lo stesso per entrambe le richieste.

Se dopo l'apertura l'utente non richiede la chiusura del cancello, il cancello rimarrà aperto fino a quando non viene raggiunto il tempo impostato per la chiusura automatica dopodiché si chiuderà automaticamente.

Se il cancello è:

- aperto, tutti i led del dispositivo sono accesi
- chiuso, tutti i led del dispositivo sono spenti
- in movimento, led di movimento giallo lampeggia con una frequenza di 0,5 Hz

User stories corrispondenti 1, 2, 9 e 10

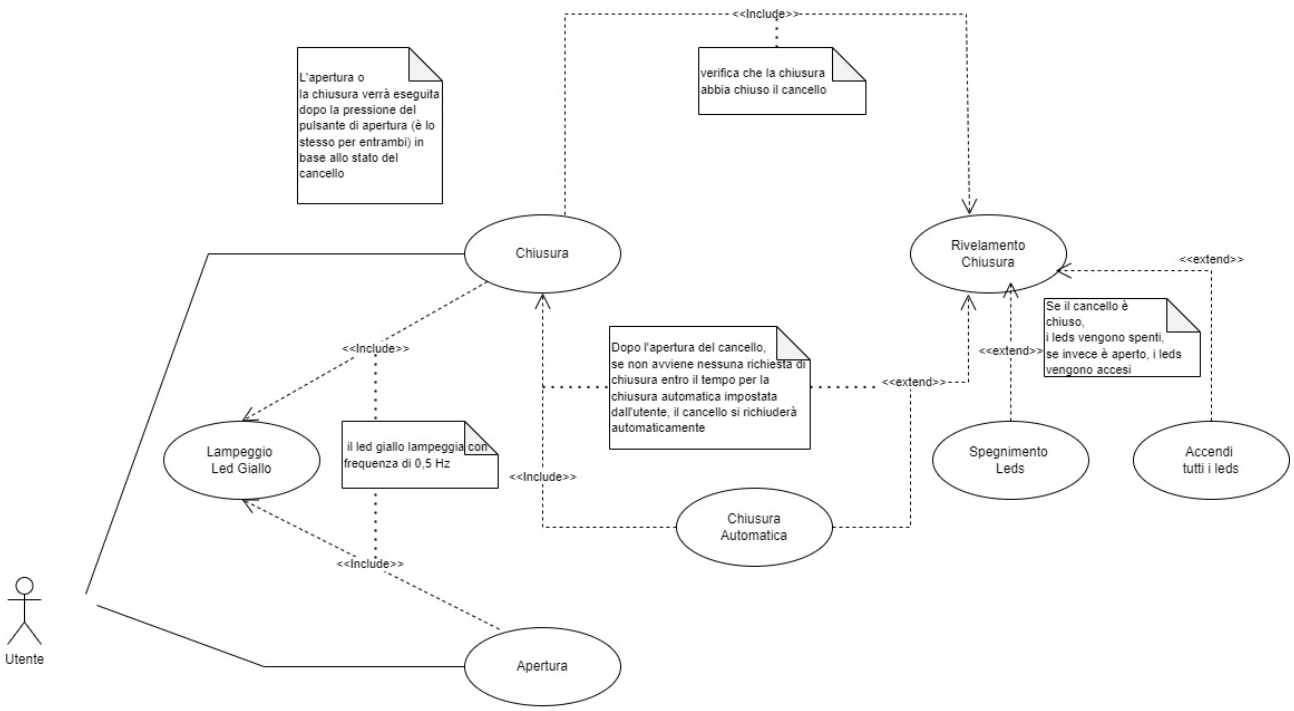

Figura 3: Funzionamento

Presenza ostacoli

Il sistema è progettato per garantire la sicurezza.

Se viene rilevato un ostacolo:

- durante la fase di chiusura, il cancello si riapre automaticamente per evitare collisioni.
- quando il cancello è chiuso o aperto, qualsiasi comando di apertura o chiusura viene ignorato, prevenendo movimenti indesiderati. Il led di presenza ostacoli lampeggia per 30 secondi con una frequenza di 1 Hz per avvertire l'utente.

> Nota: Se il cancello si riapre a causa della presenza dell'ostacolo, lampeggia il led di movimento (giallo) con una frequenza di 0,5 Hz

User stories corrispondenti 5 e 6.

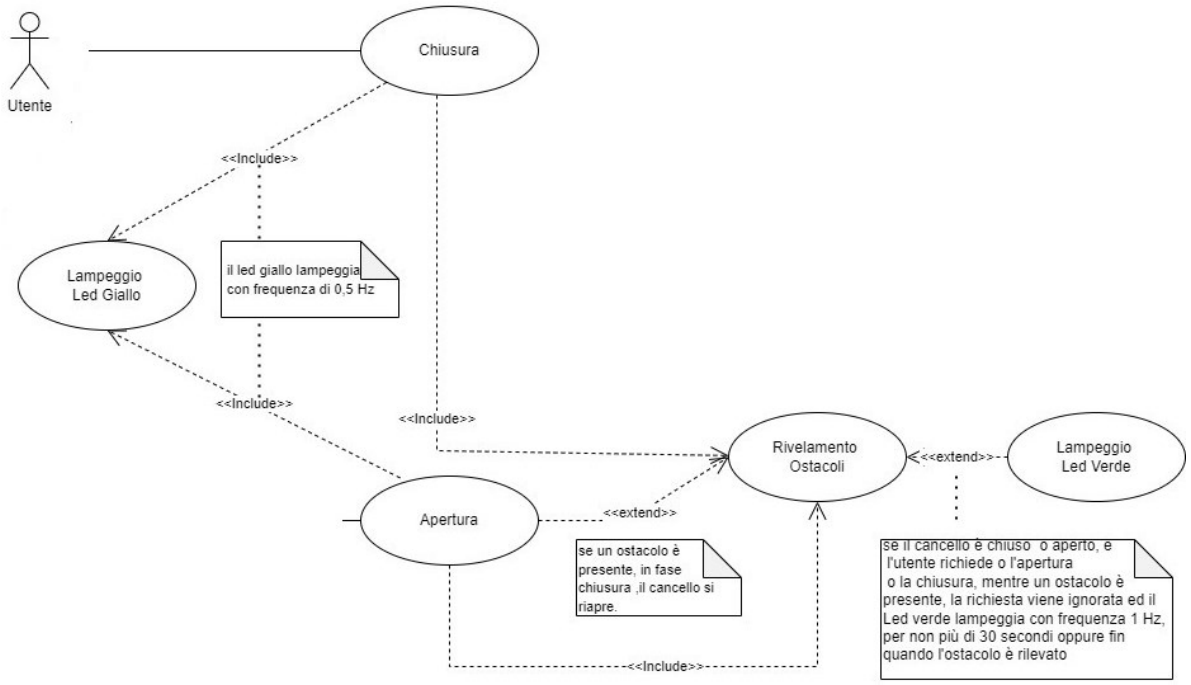

Figura 4: Presenza ostacoli

Verifica chiusura

Il sistema monitora continuamente lo stato del sensore di chiusura:

- Se questo sensore non si attiva entro la fine del tempo di lavoro, il dispositivo entra in uno stato di errore. Se dopo ulteriori 10 secondi il sensore di chiusura non è attivo, viene segnalato l'errore dall'accensione del led di errore. Si esce dallo stato di errore quando il sistema viene riavviato.

> Nota: Il cancello viene considerato correttamente chiuso solo quando il sensore di chiusura è attivo.

User stories corrispondenti 7, 8 e 12

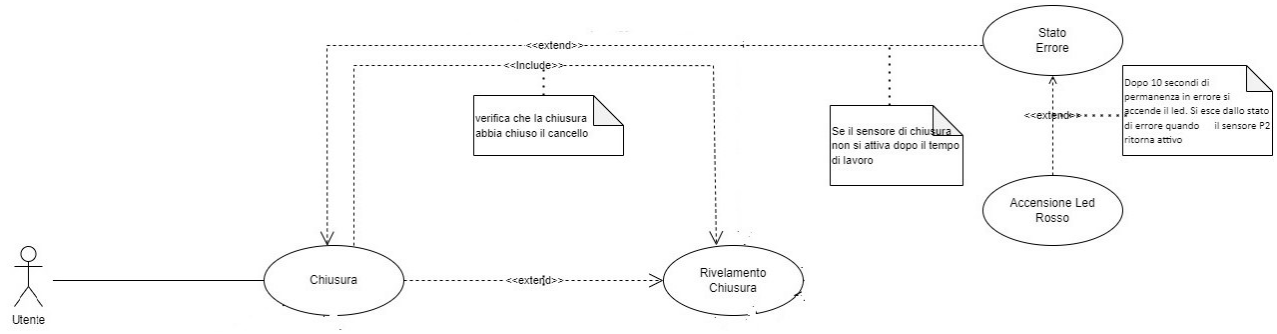

Figura 5: Verifica chiusura

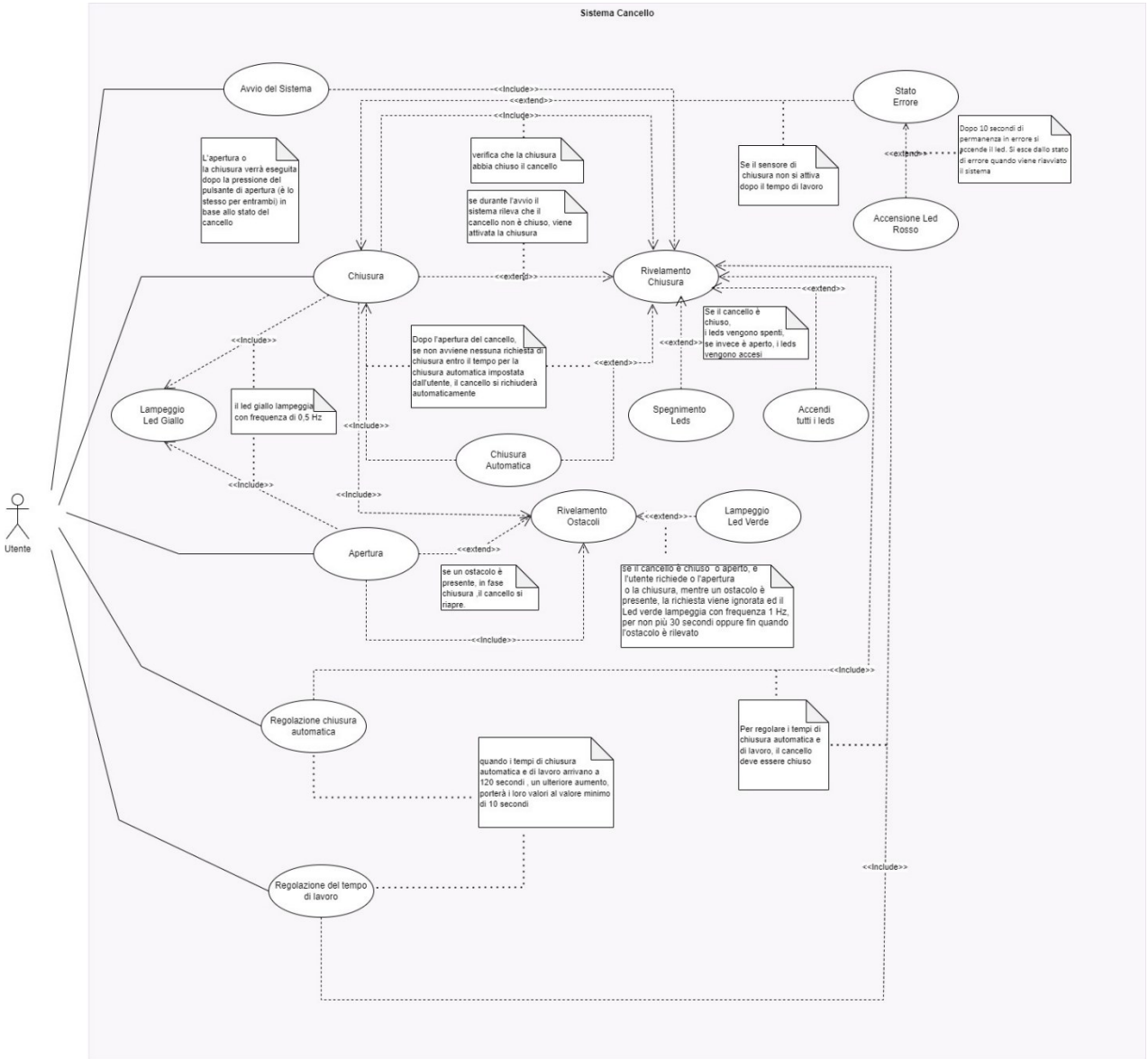

Figura 6: Use Case Diagram generale

1.3 Activity Diagrams

Gli activity diagrams rappresentano il flusso di attività o azioni che si ha nel sistema. Sono stati prodotti molteplici di questi diagrammi, in maniera tale da rappresentare scenari differenti e tipici utilizzando gli elementi messi a disposizione dalla sintassi UML.

Apertura/Chiusura

Assumiamo che il cancello sia inizialmente chiuso o in fase di chiusura. Il diagramma qui riportato descrive la sequenza che consente la riapertura e la chiusura del cancello. L'utente può aprirlo premendo il pulsante B1. Una volta attivata la fase di apertura, si accende il led di movimento (led giallo) lampeggiando con una frequenza di 0.5 Hz. Prima che il cancello si apra completamente, può essere chiuso premendo nuovamente il pulsante B1, avviando così la fase di chiusura. Se il pulsante B1 non viene premuto, il cancello si apre completamente e inizia una fase di controllo per la chiusura automatica. Se il tempo per la chiusura automatica scade e il pulsante B1 non viene premuto, il cancello si chiude automaticamente. Durante la chiusura, il led giallo lampeggia con una frequenza di 0.5 Hz fino a quando il sensore di chiusura rileva che il cancello è completamente chiuso. Se il cancello è aperto e l'utente preme il pulsante B1 prima che scada il tempo per la chiusura automatica, si avvia la fase di chiusura del cancello.

Nota: Se prima della scadenza del tempo per la chiusura automatica viene rilevata una presenza ostacoli, il conteggio del tempo viene resettato, ripartendo quando non ci sono più ostacoli.

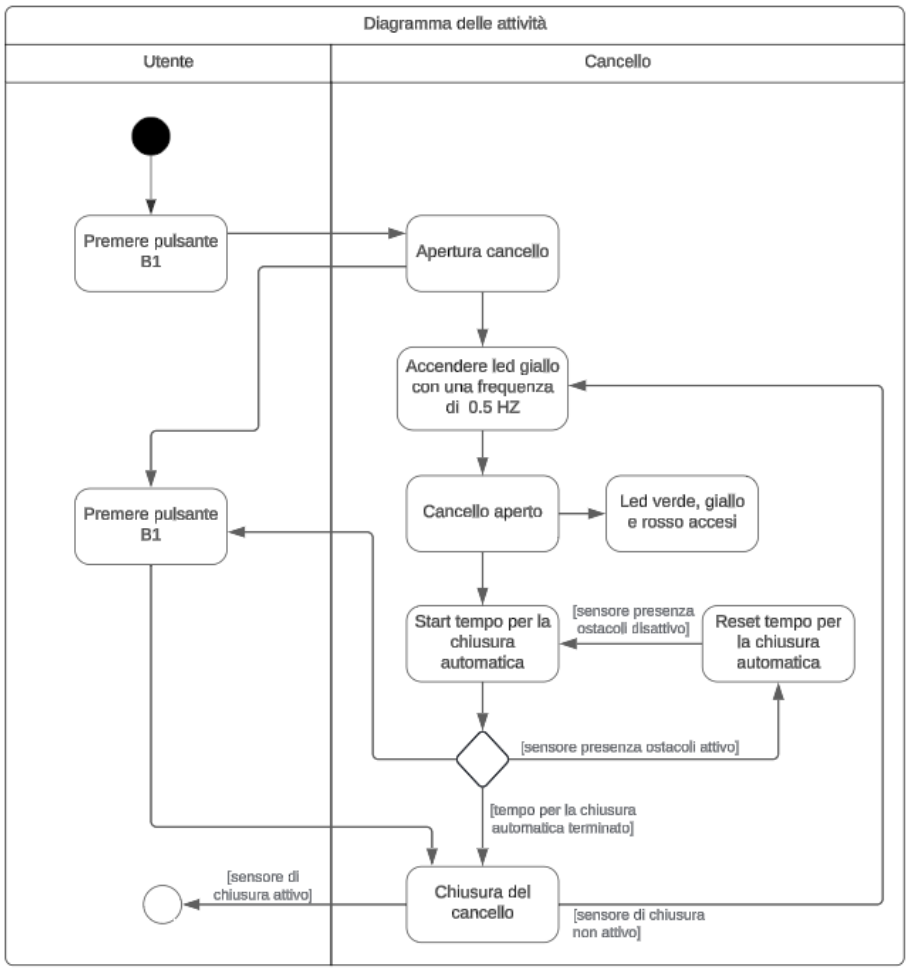

Figura 7: Apertura/chiusura

Rilevamento ostacoli

Assumiamo che il cancello sia inizialmente aperto o in fase di apertura. L'utente premendo il pulsante B1 attiva la fase di chiusura, accendendo il led di movimento (led giallo) con una frequenza di 0.5 Hz. Se il sensore di presenza ostacoli P1 rileva un ostacolo durante la fase di chiusura, il cancello si riapre istantaneamente per motivi di sicurezza. In questa situazione, se l'utente preme il pulsante B1, il comando viene ignorato dal sistema e si accende il led di presenza ostacoli (led verde) che lampeggia per 30 secondi con la frequenza di 1Hz. Una volta che l'ostacolo non è più rilevato, il led verde si spegne, ripristinando la normale funzionalità del cancello.

Nota: Se il cancello si riapre a causa della presenza dell'ostacolo, lampeggia il led di movimento (giallo) con una frequenza di 0,5 Hz, il led di rilevamento ostacoli (verde) si accende solo a cancello rispettivamente aperto o chiuso.

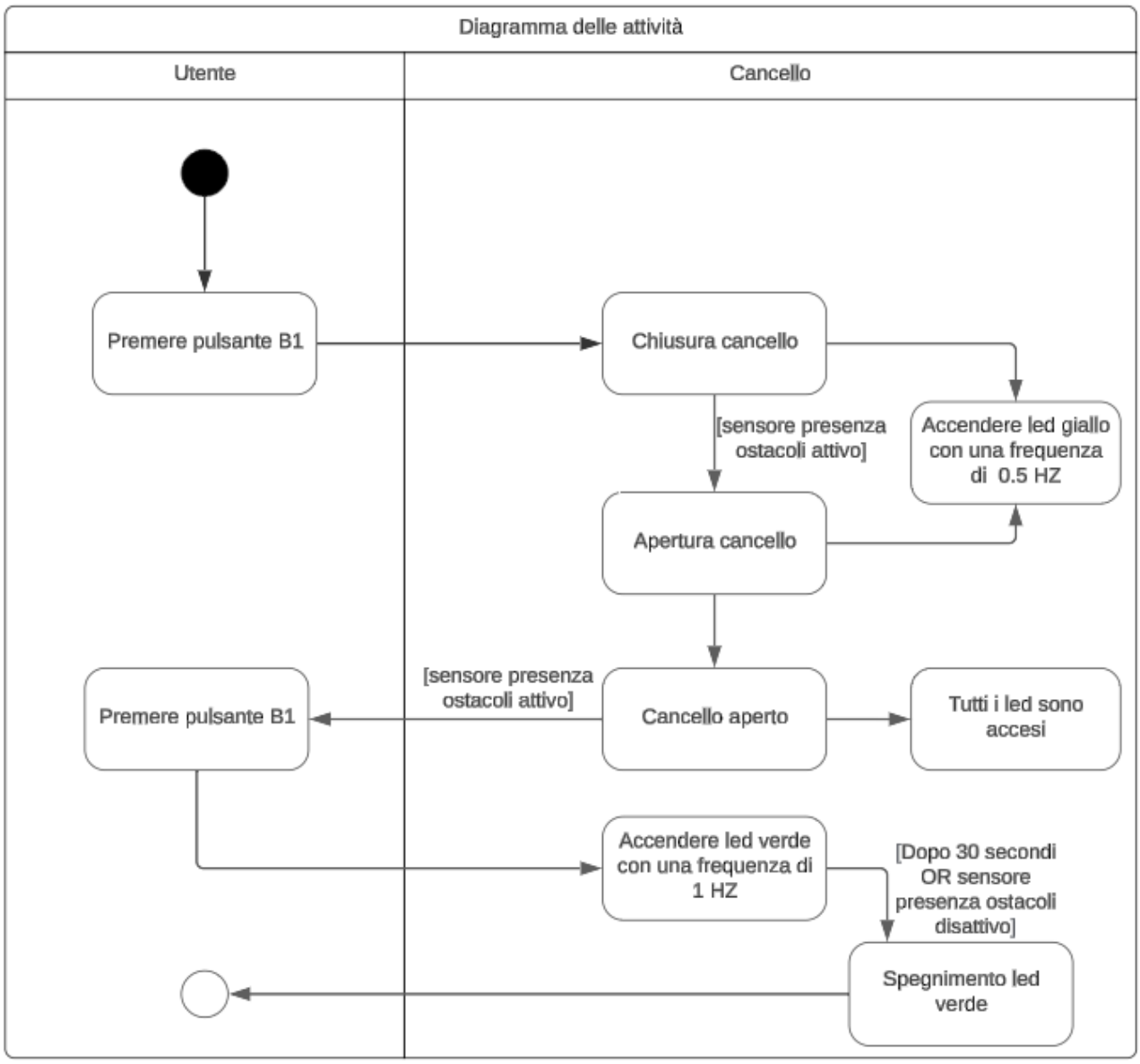

Figura 8: Rilevamento ostacoli

Configurazione dei tempi di operazione del cancello

Assumiamo che il cancello sia inizialmente chiuso. Alla pressione del pulsante per la chiusura automatica (B2), l'utente può regolare il tempo di chiusura automatica del cancello, incrementandolo di 10 secondi ad ogni pressione del pulsante. Se il tempo di chiusura del cancello supera i 120 secondi, viene resettato a 10 secondi. Alla pressione del pulsante B3 del tempo di lavoro (durata delle fasi di apertura e di chiusura del cancello), l'utente può impostare il tempo di lavoro del cancello, incrementandolo di 10 secondi ad ogni pressione del medesimo pulsante. Anche in questo caso, se il tempo di lavoro del cancello supera i 120 secondi, viene resettato a 10 secondi. Il nodo fork presente all'interno del diagramma consente contemporaneamente di valutare la selezione attuale, o di premere nuovamente i pulsanti B2 o B3 per ulteriori regolazioni. L'utente può ripetere l'operazione di configurazione fino a quando non è soddisfatto, a patto che il cancello sia chiuso.

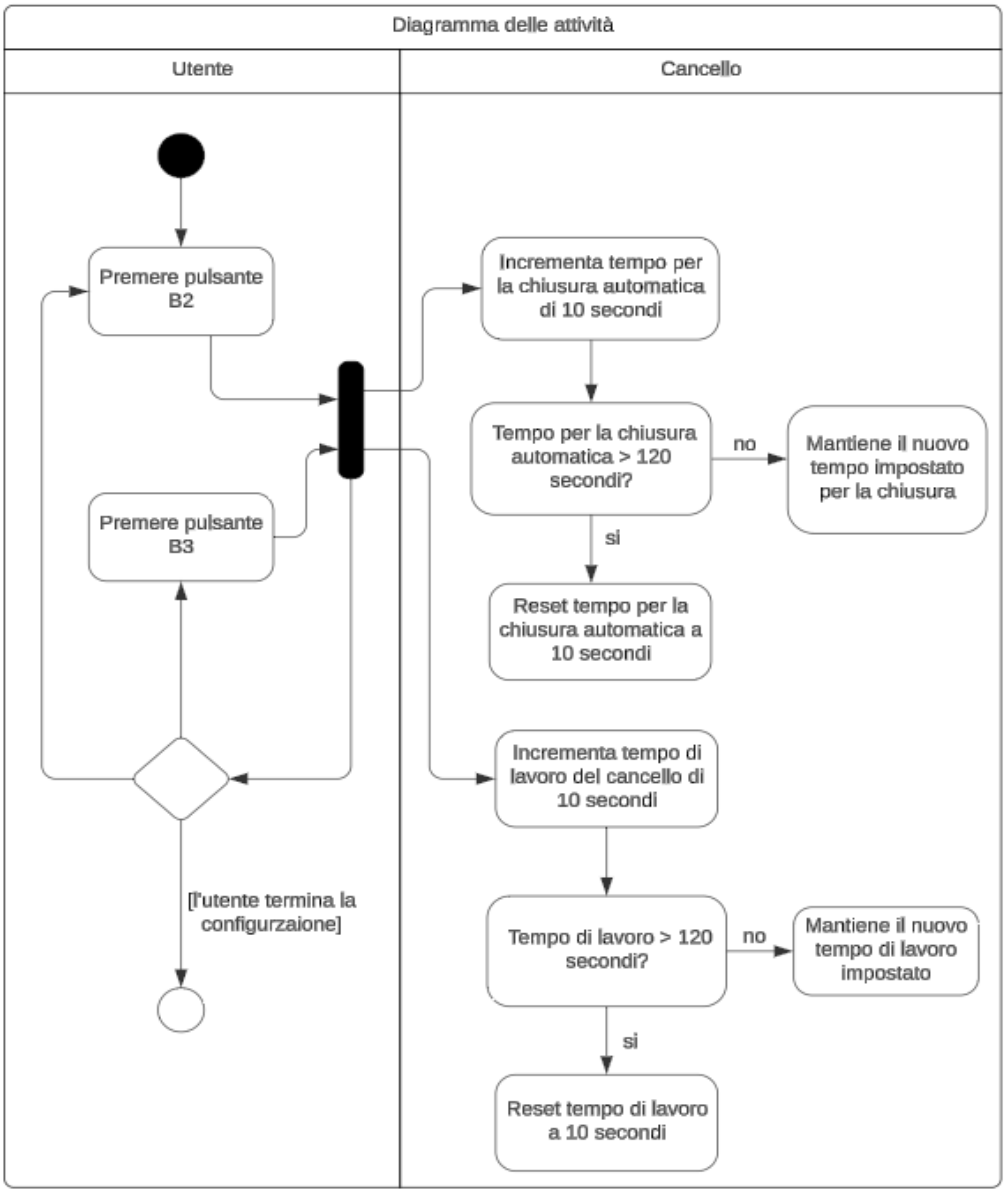

Figura 9: Configurazione dei tempi di operazione del cancello

Accensione del sistema

Assumiamo che il sistema sia inizialmente spento. Il diagramma qui riportato descrive la sequenza che consente di verificare lo stato del sensore di chiusura durante l'accensione del sistema. L'utente accende il sistema, avviando la fase di verifica sullo stato del sensore di chiusura P2. Se il sensore di chiusura non è attivo, viene avviata la fase di chiusura del cancello, durante la quale si accende il led di movimento (led giallo) lampeggiando con una frequenza di 0.5 Hz. Una volta che il sensore di chiusura rileva che il cancello è chiuso, tutti i led si spengono e il dispositivo è pronto per l'utilizzo.

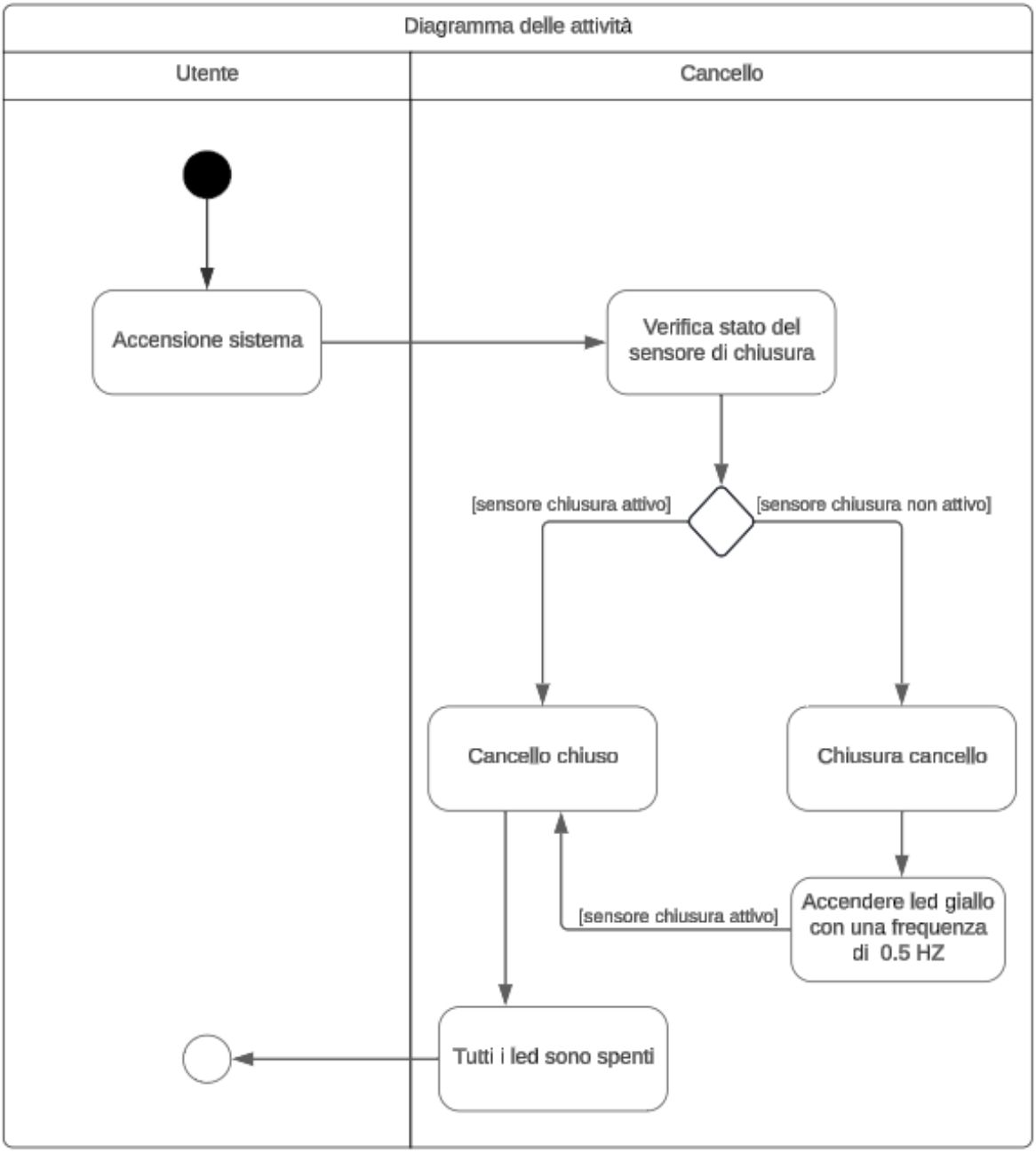

Figura 10: Accensione del sistema e verifica sensore chiusura

Verifica corretta chiusura del cancello

Assumiamo che il cancello sia inizialmente chiuso o in fase di chiusura. Il diagramma qui riportato descrive la sequenza che consente di verificare la corretta chiusura del cancello. L'utente preme il pulsante B1, attivando l'apertura del cancello e accendendo il led di movimento (led giallo) con una frequenza di 0.5 Hz. Una volta terminato il tempo di lavoro, il sistema verifica la corretta chiusura del cancello. Se il sensore di chiusura è attivo, il cancello viene considerato chiuso e tutti i led si spengono. Se il sensore non è attivo entro 10 secondi dalla fine del tempo di lavoro, viene generato un errore, indicando un problema che richiede assistenza e si accende il LED rosso.

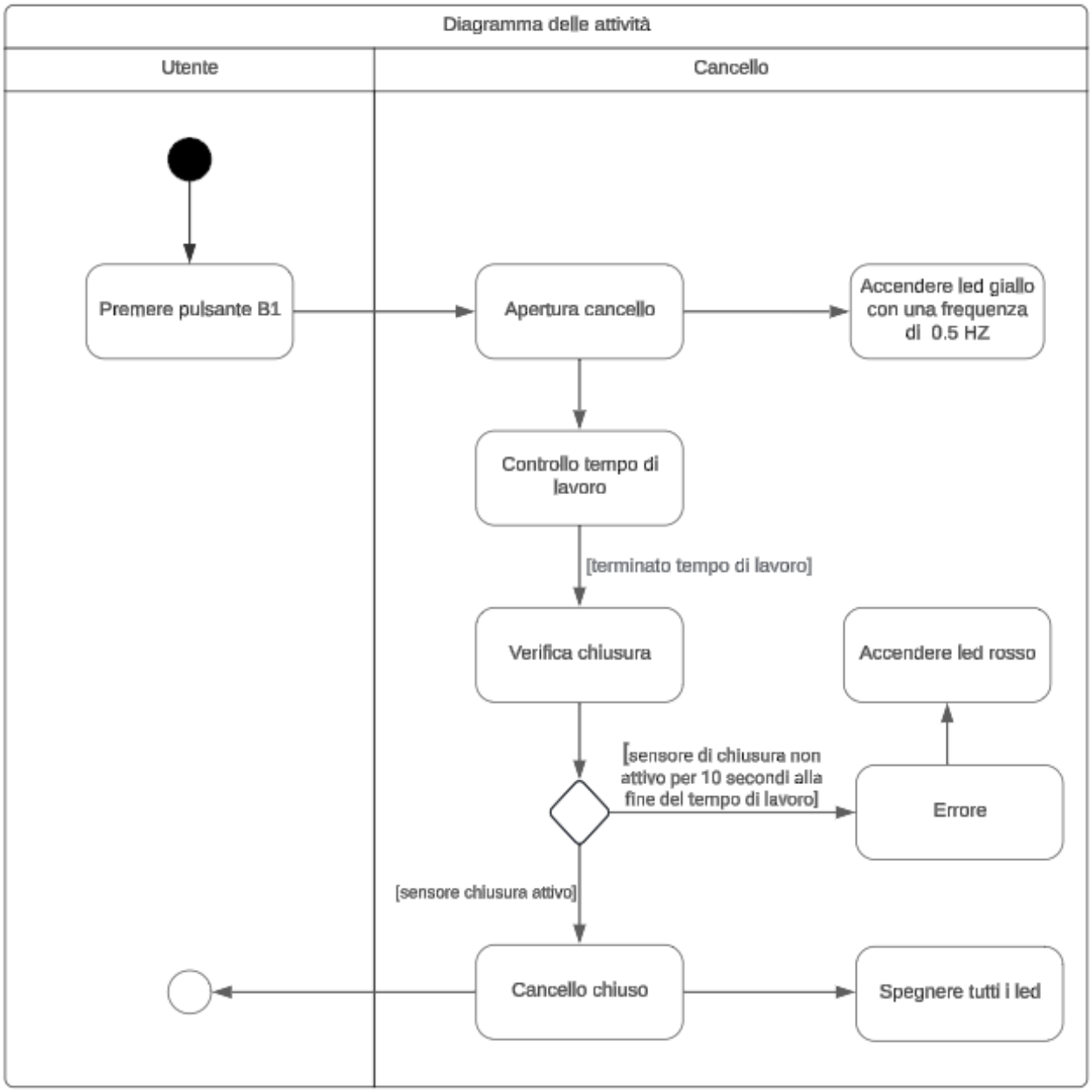

Figura 11: Verifica corretta chiusura del cancello

1.4 State diagram

Lo *state diagram* descrive i diversi stati che un oggetto può assumere nel sistema, e il cambiamento da uno stato all'altro dovuto a determinati eventi o condizioni.

Il diagramma, mostrato in figura 12, presenta sei stati principali:

- **Check**
- **Close**
- **Open**
- **Update**
- **Obstacle**
- **Error**

Appena avviata, la **macchina** si trova nello stato di **Check** che serve per controllare che all'avvio del sistema il cancello sia completamente chiuso, in caso contrario passare allo stato di **Close** o **Open** in base alla situazione.

Per poter effettuare l'apertura o la chiusura del cancello bisogna che si verifichi una doppia condizione: *non deve essere presente un ostacolo davanti al sensore P1 e il pulsante di apertura/chiusura deve essere premuto*. Il verificarsi di tale evento ci permette di passare dallo stato di **Close** allo stato di **Open** e viceversa.

Lo stato di **Close** rappresenta un superstato al cui interno è presente la logica di chiusura del cancello. Questo superstato è composto da:

- Uno stato **Closing** che si occupa della fase di chiusura del cancello. Il sistema permane in questo stato fino al termine del tempo di lavoro avvisando l'utente della chiusura tramite il lampeggio di un led. La presenza di un ostacolo davanti al sensore P1 fa passare il sistema nello stato di **Open**.
- Uno stato **Closed** che indica la chiusura completa del cancello, in questo stato tutti e tre i led sono spenti. Il sistema permane in questo stato fino al verificarsi della doppia condizione.

Lo stato **Open** è un superstato al cui interno è presente la logica di apertura del cancello. Questo superstato è composto da:

- Uno stato di **Opening** che si occupa dell'apertura del cancello. Il sistema permane in questo stato fino al temine del tempo di lavoro avvisando l'utente dell'apertura tramite il lampeggio di un led.
- Uno stato di **Opened** che indica l'apertura completa del cancello avvisando l'utente tramite l'attivazione di tre led. Il sistema permane in questo stato fino al termine del tempo di chiusura. In caso di presenza di un ostacolo il tempo per la chiusura viene resettato. Al termine del tempo per la chiusura il sistema passa allo stato **Close**.

Lo stato **Update** è un superstato che permette all'utente di modificare il tempo di lavoro e il tempo per la chiusura solo quando il cancello è completamente chiuso. Questo superstato è formato da

due sistemi che lavorano parallelamente. Al raggiungimento dei 120 secondi si riportano i timer a 10 secondi.

Lo stato di **Obstacle** è un superstato che si occupa di notificare all'utente, quando quest'ultimo preme il pulsante di apertura, tramite un apposito led, che davanti al sensore P1 è presente un ostacolo.

Lo stato **Error** è un superstato che si occupa di segnalare all'utente tramite un apposito led che il cancello non si è chiuso correttamente. Il sistema permane in questo stato fino all'intervento manuale dell'utente.

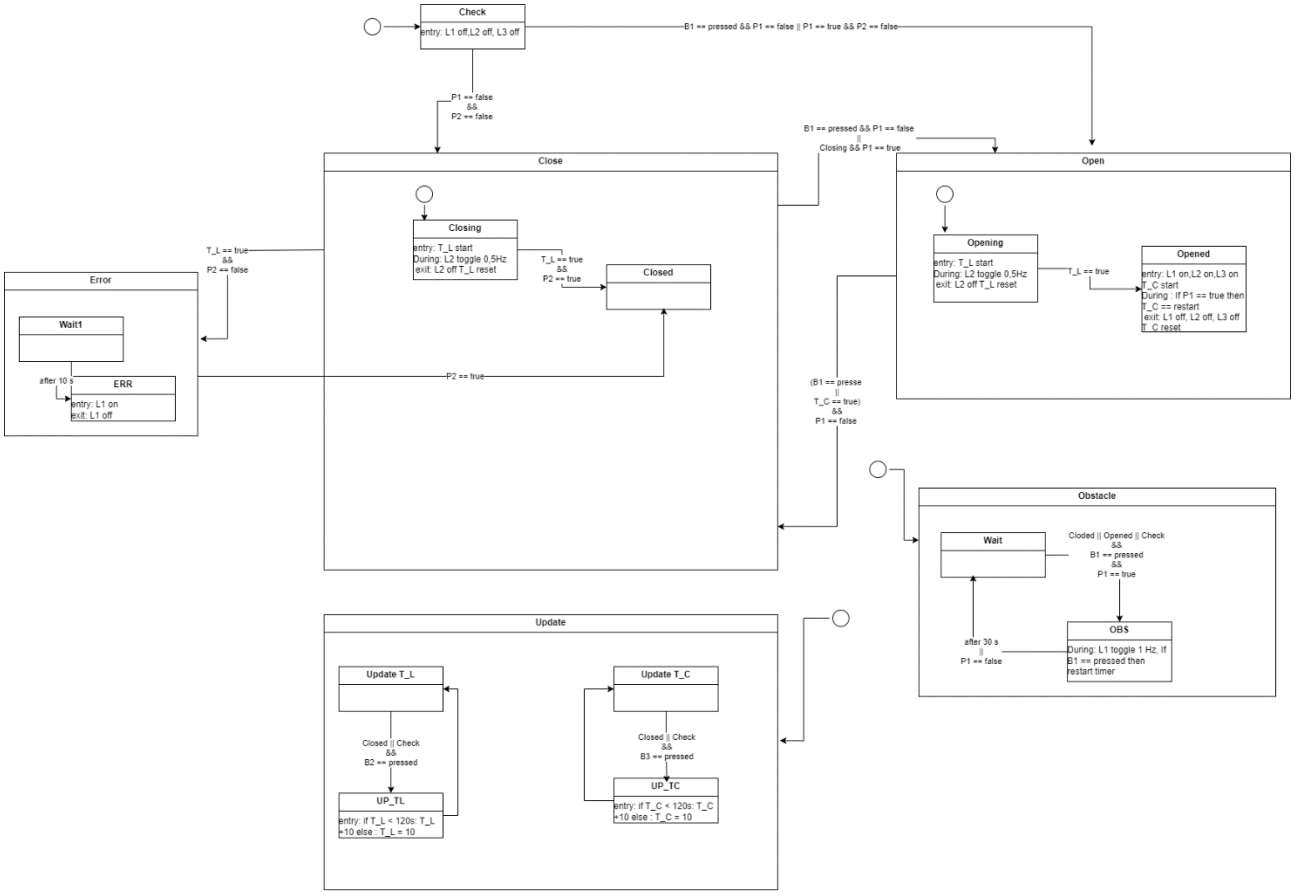

Figura 12: State diagram

Modellazione del sistema nell’ambiente Simulink

Questo capitolo è dedicato alla modellazione del sistema nell’ambiente di Simulink tramite Stateflow.

Stateflow è uno strumento potente all'interno di Simulink di MATLAB, ideale per la modellazione di sistemi complessi attraverso diagrammi di stato. È particolarmente efficace nella gestione di logiche basate su stato, condizioni decisionali e comportamenti che cambiano nel tempo. Stateflow permette di definire stati, transizioni condizionate e azioni specifiche all'ingresso, all'uscita o durante la permanenza in uno stato, facilitando la rappresentazione grafica e la simulazione di sistemi dinamici complessi.

2.1 Descrizione del modello Stateflow

In una prima fase, è stata realizzata una copia quasi identica del diagramma di stato descritto nel capitolo precedente. Successivamente, sono state apportate alcune piccole modifiche risultate necessarie durante la verifica del corretto funzionamento del modello in Simulink o a seguito dei test effettuati. Tuttavia, la struttura generale del modello è rimasta sostanzialmente invariata.

Control System Cancello

Il sistema di controllo del cancello elettrico a battente presenta le seguenti componenti:

- Un’istanza del bottone di apertura e chiusura del cancello;
- Un’istanza del bottone del tempo di lavoro;
- Un’istanza del bottone per la chiusura automatica;
- Un’istanza per il funzionamento del cancello;
- Un’istanza per l’accensione/spegnimento e lampeggio del led giallo;
- Un’istanza per l’accensione/spegnimento e lampeggio del led verde;

Gli input forniti al cancello sono:

- P1: segnale che indica la presenza/assenza di un ostacolo;
- P2: segnale che indica la presenza/assenza del cancello chiuso;
- B1: segnale che indica pressione/rilascio del pulsante di apertura del cancello;
- B2: segnale che indica pressione/rilascio del pulsante del tempo di lavoro;
- B3: segnale che indica pressione/rilascio del pulsante per la chiusura automatica.

Nota: Nella traccia viene indicato B2 come pulsante per il la chiusura automatica e B3 come pulsante del tempo di lavoro. In questo progetto sono stati invertiti.

Gli output invece sono:

- L1: segnale che determina l’accensione/spegnimento del LED rosso di allarme;
- L2: segnale che determina l’accensione/spegnimento del LED giallo;

- L3: segnale che determina l'accensione/spegnimento del LED;
- L2_Blink: segnale che determina il lampeggio del LED giallo quando il cancello è in movimento;
- L3_Blink: segnale che determina il lampeggio del LED giallo quando presente un ostacolo a cancello chiuso o aperto.

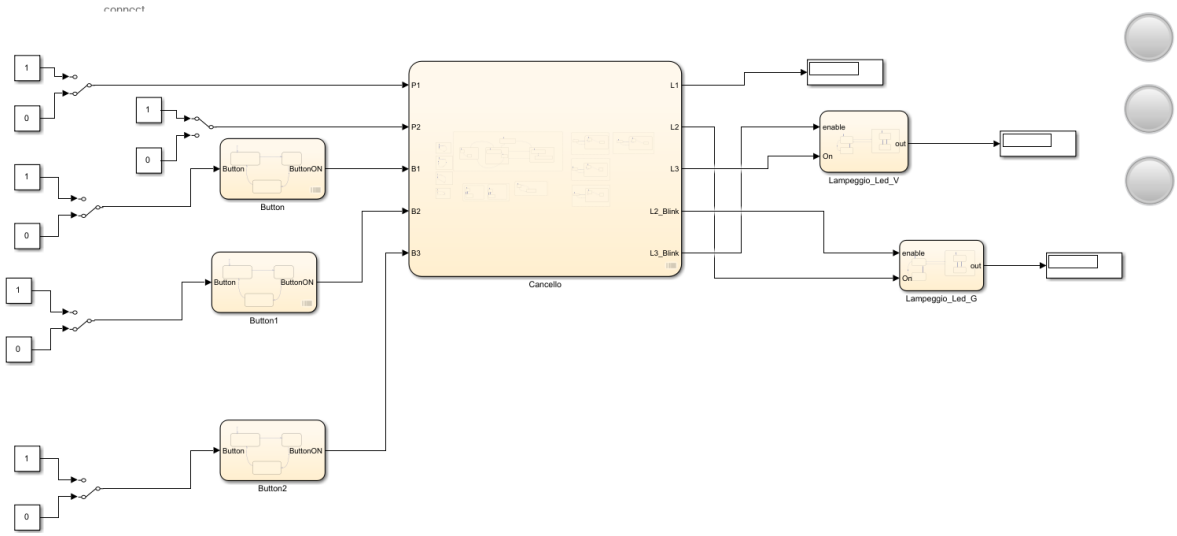

Figura 13:Stateflow cancello automatico

Button chart

Questo modulo è stato ideato per fare in modo che l'input relativo alla pressione di un pulsante avvenga dopo aver premuto e poi rilasciato il pulsante. Il modulo viene utilizzato nel sistema per la pressione del pulsante di apertura/chiusura e per i pulsanti di regolazione della chiusura automatica e del tempo di lavoro. In sintesi, il modulo assicura che ogni comando dato dai pulsanti venga considerato valido solo dopo che il pulsante è stato premuto e poi rilasciato.

Dopo aver descritto la logica passiamo alla descrizione del chart che illustra il funzionamento. Iniziamo con l'esporre il significato delle variabili utilizzate:

Tipo	Nome	Valore	Descrizione
Input data	Button		Variabile di input relativa alla pressione del bottone
Output data	ButtonOn		Variabile di output che indica l'effetiva pressione del pulsante
Constant data	Pressed	True	Constante che indica la pressione del bottone
Constant data	Released	False	Constante che indica il rilascio del bottone

Il modulo di divide in 3 stati principali:

- **Wait**
- **Pressed**
- **LongPressed**

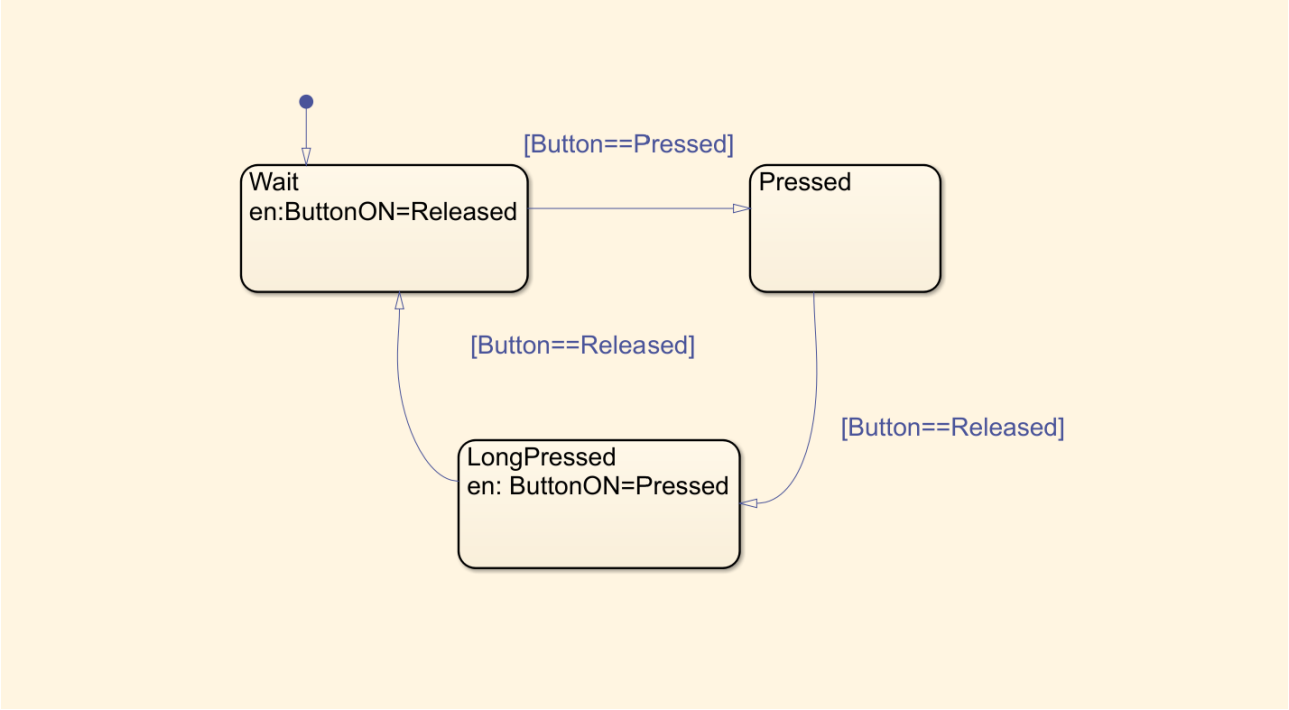

Figura 14:Button Chart

A questo punto, iniziamo a descrivere il funzionamento del modulo:

Esso parte dallo stato di **Wait**, alla cui attivazione *ButtonON* viene impostato a *Released*, dopodiché alla pressione del bottone segnalata mediante la variabile di input *Button* la transizione farà confluire nello stato **Pressed** dove si rimane fino a quando il bottone non viene rilasciato, confluendo nello stato **LongPressed** in cui *ButtonON* verrà impostata a *Pressed,* notificando che il pulsante risulta effettivamente premuto, infine confluirà nello stato iniziale.

Lampeggio led chart

In questa sezione discuteremo dell'implementazione del modulo *Lampeggio_Led_X* che permette di dare l'effetto lampeggio al led. Il modulo si divide in 3 stati principali:

- **off**
- **on**
- **blink**

La macchina si trova inizialmente nello stato off, in questo contesto l'uscita del modulo sarà pari a zero, ciò corrisponde ad un led spento. Ci sono due segnali di input: *enable ed* On. Quando viene attivato il segnale On si confluisce nello stato on dove l'uscita viene impostata a uno, indicando l'accensione del led. Quando viene attivato il segnale *enable* si confluisce nello stato *blink,* in questo

stato una transazione farà in modo che i due sottostati si alternino con un periodo costante. Analizziamo il funzionamento di tali sottostati:

- **blink_on**
- **blink_off**

Il sottostato **blink_on** è il sottostato iniziale del superstato **blink** e mantiene in uscita il segnale out ad uno, questo significa che il led sarà inizialmente acceso e si spegnerà soltanto dopo period millisecondi.

Il sottostato **blink_off** è uno stato che possiamo definire di attesa in cui il valore dell'uscita è tenuto a zero per **period** millisecondi in modo da poter realizzare l'effetto di lampeggio.

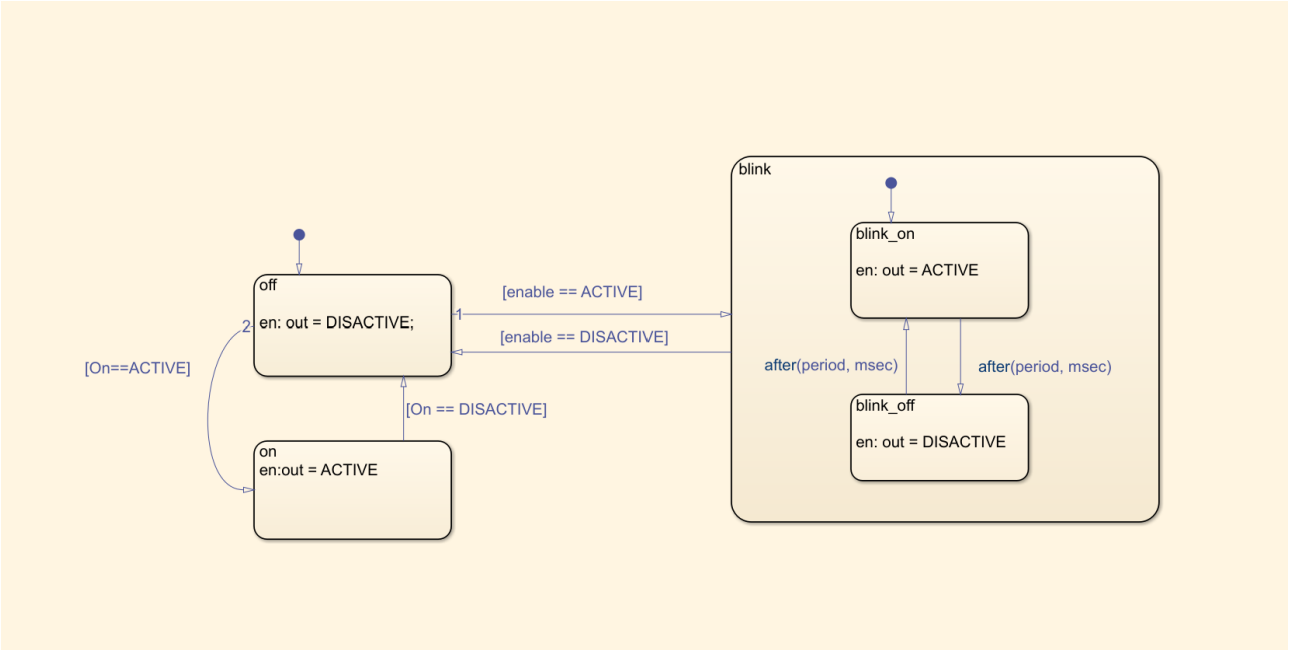

Figura 15: Lampeggio led chart

Tipo	Nome	Valore	Descrizione
Input data	*On*		Variabile di input per abilitare l'accensione constate del led
Input data	*enable*		Variabile di input per abilitare il lampeggio del led
Output data	*out*		Variabile di output per controllare il led
Constant data	*period*	1000/2000	Constante che indica il periodo di lampeggio 1000 per il led verde e 2000 per il led giallo
Constant data	*ACTIVE*		Constante per segnalare l'attivazione del led
Constant data	*DISACTIVE*		Constante per segnalare la disattivazione del led

Cancello chart

Dopo aver illustrato i componenti di base, proseguiamo con la presentazione del chart che rappresenta il cuore del sistema. In particolare, iniziamo con l'esporre il significato delle variabili utilizzate, come mostra la seguente tabella:

Tipo	**Nome**	**Valore**	**Descrizione**
Input data	*P1*		Variabile di input per segnalare la presenza di un ostacolo
Input data	*P2*		Variabile di input per segnalare la chiusura del cancello
Input data	*B1*		Variabile di input per l'apertura/chiusura del cancello
Input data	*B2*		Variabile di input per la modifica del tempo di lavoro
Input data	*B3*		Variabile di input per la modifica del tempo di chiusura
Output data	*L1*		Variabile di output per il led rosso
Output data	*L2*		Variabile di output per il led giallo
Output data	*L3*		Variabile di output per il led verde
Output data	*L2_Blink*		Variabile di output per far lampeggiare il led giallo
Output data	*L3_Blink*		Variabile di output per far lampeggiare il led verde
Constant data	*OFF*	false	Constante con valore vero
Constant data	*ON*	true	Constante con valore falso
Local data	*Timer_T_C*	10	Variabile locale che indica il tempo di chiusura automatica
Local data	*Timer_T_L*	10	Variabile locale che indica il tempo di lavoro
Local data	*RESET*		Variabile locale che indica quando il timer deve resettarsi
Local data	*Start_T_C*		Variabile locale che indica quando deve partire il timer relativo al tempo di chiusura automatica
Local data	*Start_T_L_O*		Variabile locale che indica quando deve partire il timer relativo al tempo di lavoro per l'apertura del cancello
Local data	*Start_T_L_C*		Variabile locale che indica quando deve partire il timer relativo al tempo di lavoro per la chiusura del cancello
Local data	*T_L*		Variabile locale che indica quando il timer relativo al tempo di lavoro è terminato
Local data	*T_C*		Variabile locale che indica quando il timer relativo al tempo di chiusura è terminato
Local data	*T_LV*		Variabile locale che indica che il timer per il lampeggio del led verde è terminato
Local data	*Start_T_LV*		Variabile locale che indica quando il timer per il lampeggio del lev verde deve partire
Local data	*RESET_LV*		Variabile locale che indica quando il timer relativo al lampeggio del led verde deve essere resettato

Proseguiamo con la descrizione della struttura del chart, analizzando i diversi stati e le relative transazioni. Il chart è composto da 7 stati che lavorano in parallelo:

- **Cancello**
- **Update**
- **Obstacle**
- **Timer_T_L_O**
- **TImer_T_L_C**
- **Timer_C**
- **Timer_LV**

Il superstato **Cancello** si occupa del comportamento del cancello per l'apertura e la chiusura, è composto da 4 stati:

- Lo stato di **Check** che rappresenta lo stato in cui si trova il cancello appena viene acceso, imposta tutti i led spenti. In particolare:
    - Con la transazione *[B1==ON && P1==false || P1==true &&P2==false]* si passa allo stato **Open.**
    - Con la transazione *[P1==false && P2==false]* si passa allo stato **Close.**
- Lo stato di **Close** è composto a sua volta da 2 stati: **Closing** e **Closed.**
    - Lo stato iniziale è **Closing** dove avviene la chiusura del cancello. In questo stato viene fatto lampeggiare il led giallo e viene avviato il timer di lavoro. Con la transazione *[T_L==ON && P2==true]* si passa allo stato **Closed**.
    - Lo stato di **Closed** che indica che il cancello è completamente chiuso.
    - Con la transazione *[B1==ON && P1==false || in (Close.Closing) && P1==true*] si passa allo stato di Open.

  Con la transizione *[T_L==ON && P2==false]* si passa allo stato di **Error**.

- Lo stato di **Open** è composto a sua volta da 2 stati: **Opening** e **Opened**.
    - Lo stato iniziale è **Opening** dove avviene l'apertura del cancello. In questo stato viene fatto lampeggiare il led giallo e viene avviato il timer di lavoro. Con la transazione *[T_L==ON]* si passa allo stato di **Opened**.
    - Lo stato di **Opened** che indica che il cancello è completamente aperto. In questo stato si accendono tutti e 3 i led e viene avviato il timer di chiusura automatica.

  Con la transazione *[(B1==ON || T_C==true) && P1==false]* si passa allo stato di **Close.**

- Lo stato di **Error** è composto a sua volta da 2 stati: **Wait** e **ERR**.
    - Lo stato iniziale è **Wait** dove si aspetta 10 secondi che il cancello si chiuda. La transizione *[after(10,sec)]* fa passare il sistema nello stato di **ERR**.
    - Lo stato di **ERR** indica che c'è un errore. In questo stato il led rosso viene acceso.

  La transizione *[P2==true]* fa passare il sistema nello stato di **Closed**.

Il superstato **Update** si occupa di modificare il tempo di lavoro ed il tempo di chiusura automatica, è composto da 2 stati che lavorano in parallelo:

- Lo stato **Update_TL** composto a sua volta da 2 stati, si occupa alla pressione del pulsante B2 di incrementare il tempo di lavoro:
    - Lo stato iniziale **Update_T_L** è uno stato di attesa. La transazione *[(in(Cancello.Close.Closed) || in(Cancello.Check)) && B2==ON]* fa passare il sistema nello stato **UP_TL**.
    - Lo stato **UP_TL** serve per incrementare il tempo di lavoro, e lo fa tramite la funzione *Update_time_T_L()*. Effettuato l'incremento torna nello stato iniziae **Update_T_L.**
- Lo stato **Update_TC** lavora un maniera analoga al precedente.

Il superstato **Obstacle** si occupa di controllare se alla richiesta di apertura o chiusura, quando il cancello è completamente chiuso o aperto, che non sia presente un ostacolo davanti alla fotocellula. È composto da 2 stati:

- Lo stato iniziale **Wait** è uno stato di attesa. La transazione *[(in(Cancello.Close.Closed) || in(Cancello.Open.Opened) || in(Cancello.Check)) && B1==ON &&P1==true]* fa passare il sistema nello stato di **OBS**.
- Lo stato di **OBS** indica che alla richiesta di apertura o chiusura, quando il cancello è completamente chiuso o aperto, la presenza di un ostacolo davanti alla fotocellula. In questo stato viene fatto lampeggiare il led verde e viene avviato il timer per il lampeggio del led verde. La transazione *[P1==false || T_LV==ON]* fa passare il sistema nello stato **Wait**.

Il superstato **Timer_T_L_C** è uno stato che si occupa di contare il tempo di lavoro per la chiusura del cancello. È composto da 2 stati:

- Lo stato iniziale **off** che inizializza il timer a zero. La transazione *[Start_T_L_C == ON]* fa passare il sistema nello stato **Counting.**
- Lo stato **Counting** è formato da 2 stati:
    - Lo stato iniziale **Count1** che si occupa di contare il tempo di lavoro.
        - Con la transazione *[after(Time_T_L,sec)]* si passa allo stato **End**.
        - Con la transazione *[RESET == ON]* si passa allo stato **Count1**.
    - Lo stato **End** che indica che il tempo di lavoro è terminato quindi imposta il timer a uno.

  La transazione *[Start_T_L_C == OFF]* fa passare il sistema nello stato **off**.

I superstati **Timer_T_L_O, Timer_LV, Timer_T_C** funzionano in maniera analoga.

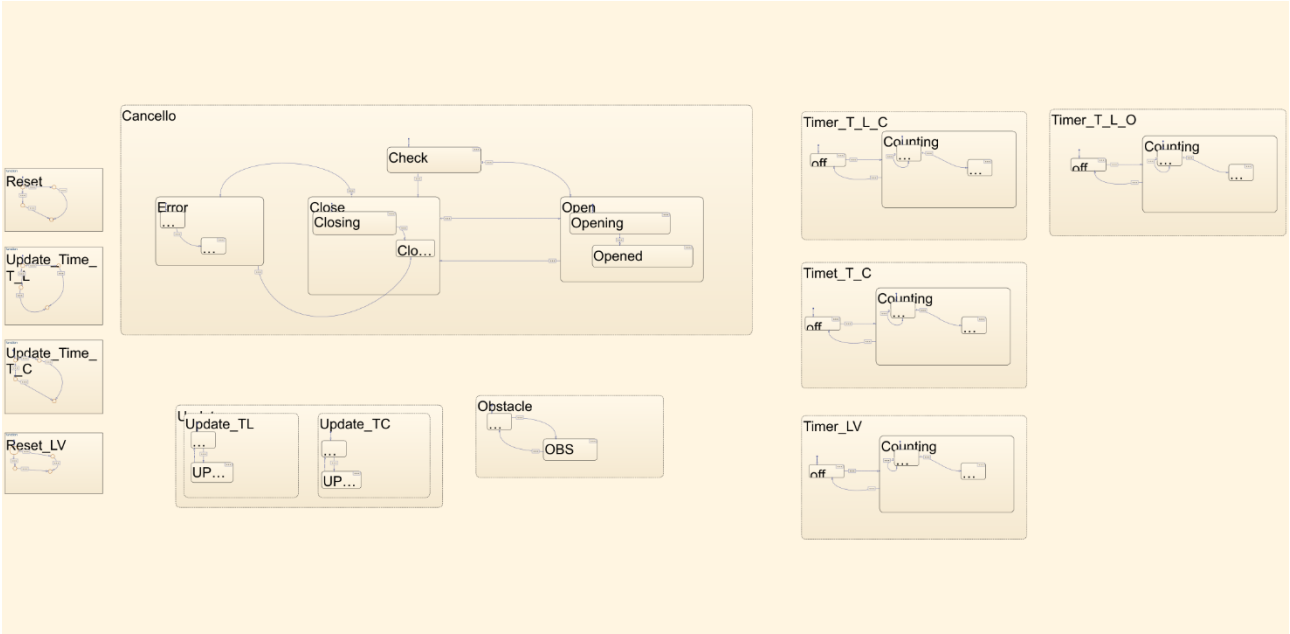

Figura 16: Cancello chart

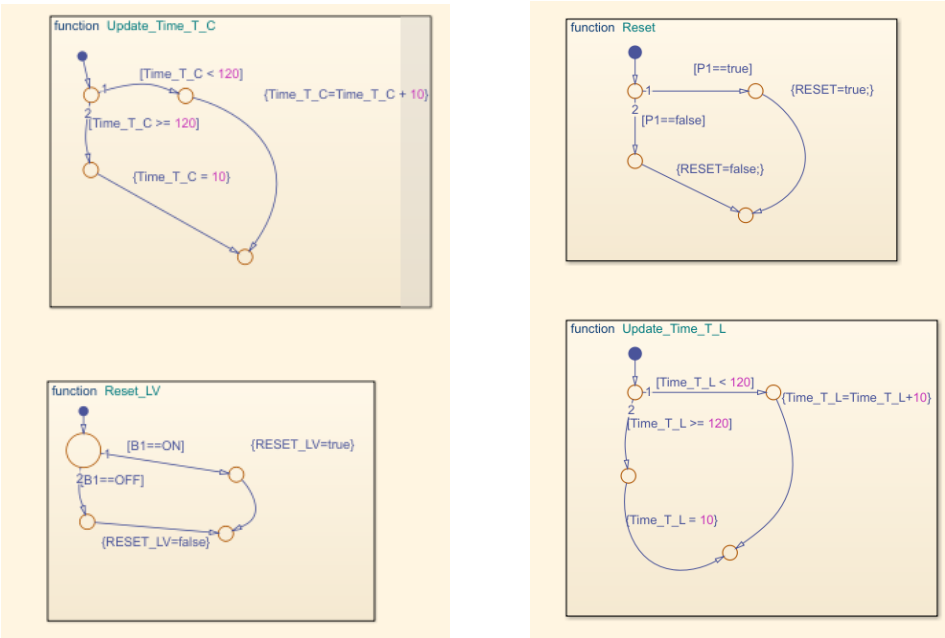

Figura 17: function

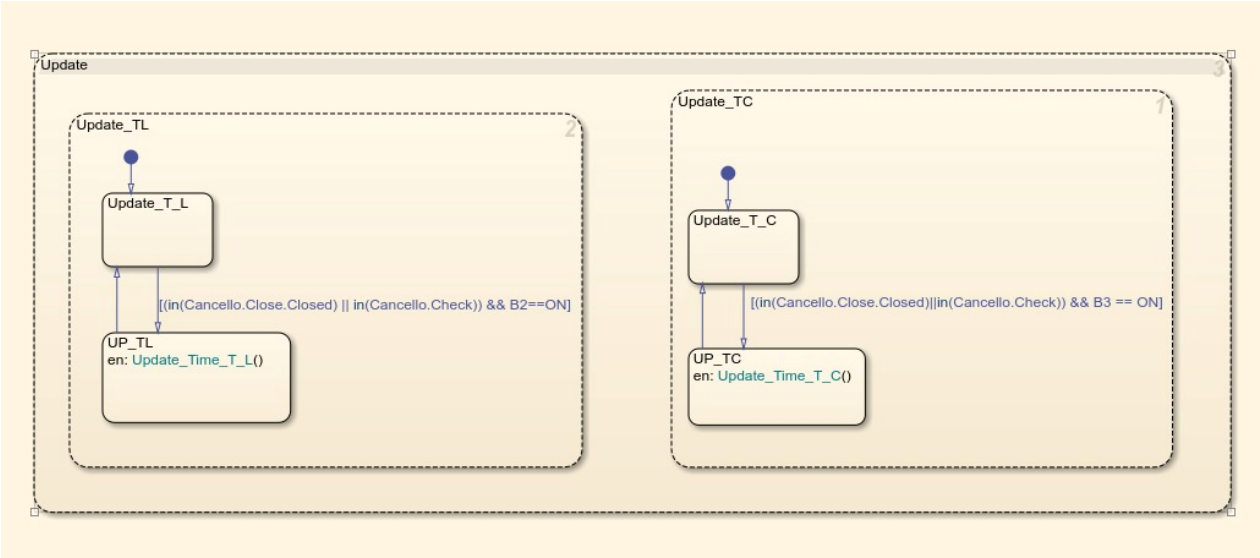

Figura 18: Update

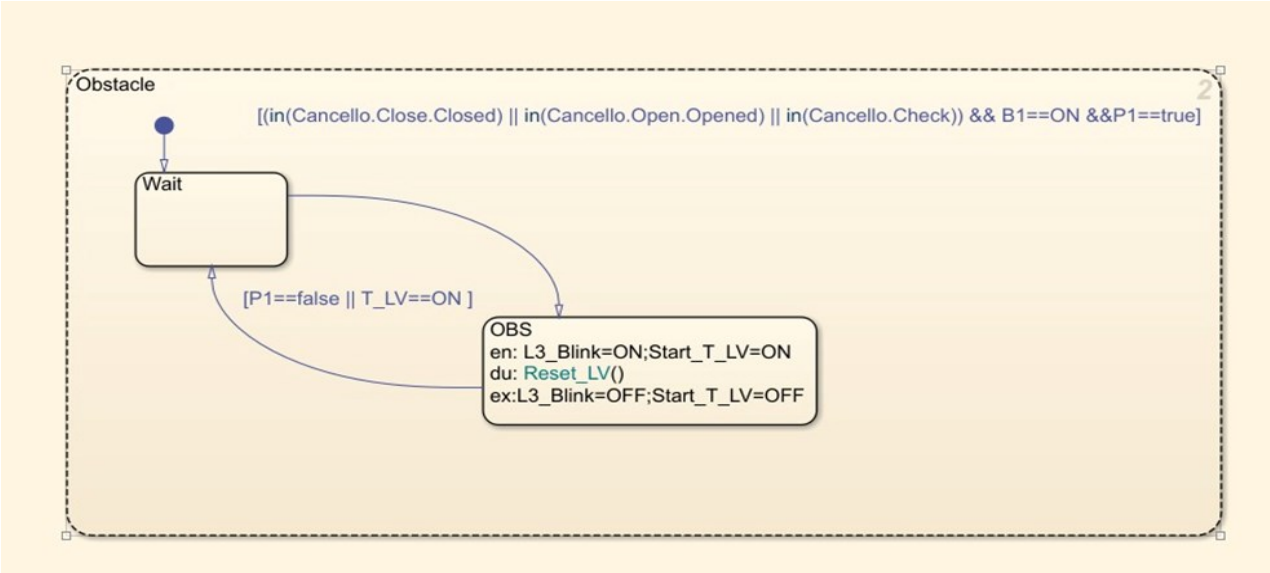

Figura 19: Ostacolo

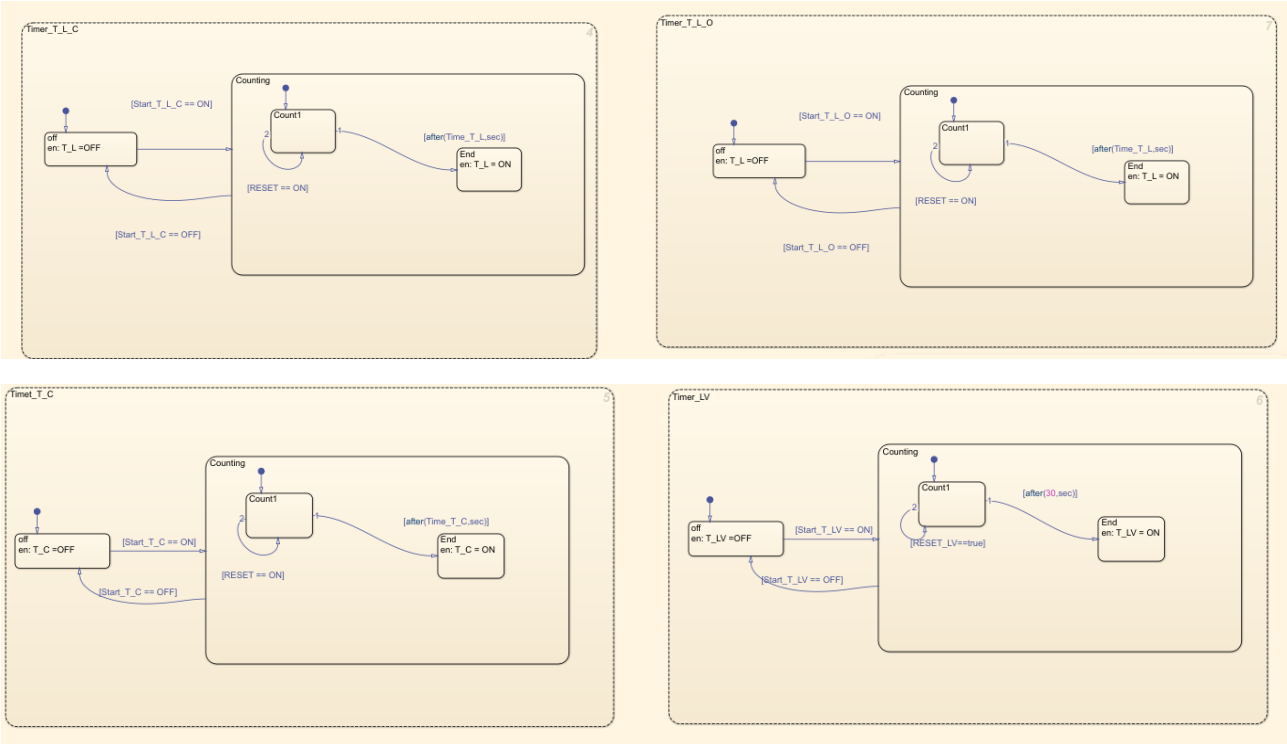

Figura 20: Timer_T_C

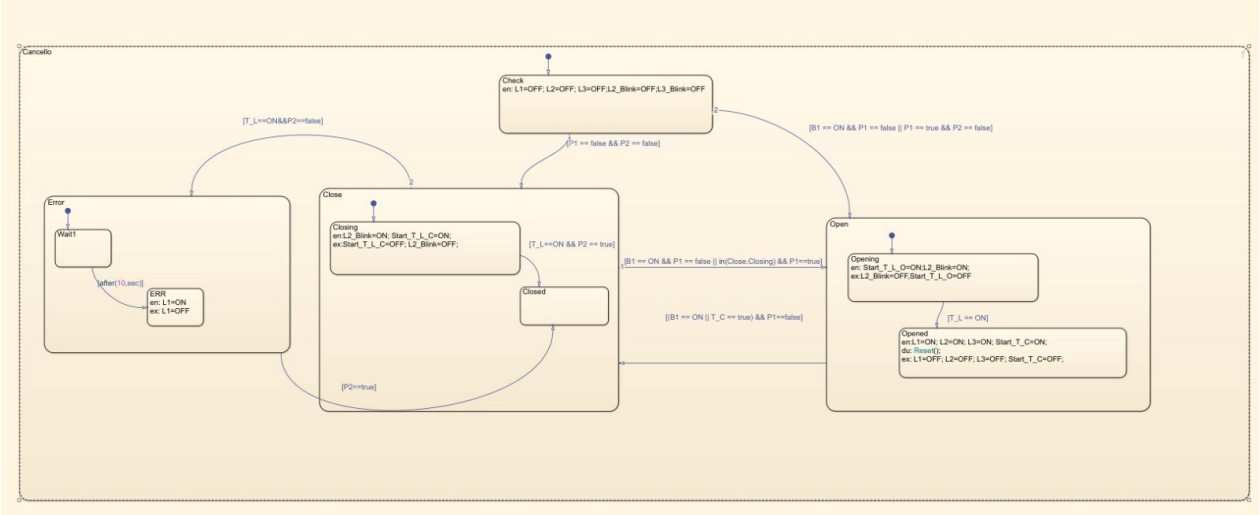

Figura 21: cancello

Validazione

Questo capitolo è dedicato alla creazione e all'esecuzione dei test sui moduli sviluppati, con l'obiettivo di verificare il corretto funzionamento del sistema. I test sono stati effettuati utilizzando Simulink Test, uno strumento integrato nell'ambiente di simulazione Simulink di MATLAB, progettato per facilitare la generazione, l'esecuzione e l'analisi dei test. Questo strumento permette di sviluppare casi di test dettagliati e replicabili, utili per valutare le prestazioni del sistema in diverse condizioni operative. Saranno esaminati i casi d'uso più rilevanti del sistema, garantendo una copertura completa delle funzionalità implementate e individuando eventuali anomalie o malfunzionamenti.

3.1 Test Led

Per verificare il corretto funzionamento dei LED in determinate condizioni, sono stati sviluppati due test: test_led_on e test_led_enable.

test_led_on

Il test **test_led_on** è stato progettato per garantire che il led si accenda e si spenga correttamente in risposta a un segnale di controllo. La procedura prevede l'avvio del test con il led inizialmente spento, l'applicazione del segnale *on=true* e la verifica dell'accensione del led. Successivamente, mantenendo il segnale *on=true* per un certo intervallo, il segnale viene modificato a *on=false*, e si verifica che il led si spenga prontamente.

Questo assicura che il led risponda correttamente ai cambiamenti del segnale on.

test_led_enable

Il secondo test **test_led_enable**, verifica che il led lampeggi per 30 secondi a una frequenza di 1 Hz quando il segnale enable è attivo. La procedura consiste nell'iniziare il test con il led spento, applicare il segnale *enable=true* e monitorare il led per assicurarsi che inizi a lampeggiare. Come già detto, il led deve lampeggiare a una frequenza di 1 Hz, quindi accendendosi e spegnendosi una volta al secondo, per un totale di 30 secondi. Durante questo periodo, si registra il comportamento del led per confermare che esegua correttamente 30 cicli di lampeggio.

Nota: Led Giallo e Led Verde hanno lo stesso funzionamento per quanto riguarda l'accensione del led.

Una differenza viene riscontrata con il lampeggio, in quanto:

- Led Verde deve lampeggiare per 30 secondi a una frequenza di 1 Hz;
- Led Giallo deve lampeggiare finché enable=true a una frequenza di 0.5 Hz.

Abbiamo inserito solo il test del led verde per non inserire test uguali.

3.2 Test Funzionamento Cancello

Per verificare il corretto funzionamento dell'apertura e chiusura del in determinate condizioni, sono stati sviluppati due test: apertura_chiusura e apertura_chiusura_automatica. Inoltre, viene testato anche il corretto funzionamento della regolarizzazione del tempo di lavoro e del tempo per la chiusura automatica.

apertura_chiusura

Nel test **apertura_chiusura**, il cancello si trova inizialmente chiuso con tutti i led spenti. Si imposta il tempo di lavoro tramite il segnale *B2*. Per garantire una corretta validazione, si aumenta gradualmente il tempo di lavoro fino a 120 secondi, per poi incrementarlo di nuovo per arrivare al valore di 10 secondi. Successivamente, si richiede l'apertura del cancello tramite il segnale *B1*, durante la quale il led giallo lampeggia per la durata del tempo di lavoro. Una volta che il cancello è aperto correttamente, tutti i led sono spenti. Successivamente, viene richiesta la chiusura del cancello tramite il segnale *B1*, durante la quale il led giallo lampeggia ancora per la durata del tempo di lavoro. Infine, il cancello è correttamente chiuso quando *P2* diventa *true* e tutti i LED sono spenti.

apertura_chiusura

Nel test **apertura_chiusura_automatica**, il cancello si trova inizialmente chiuso con tutti i led spenti. Viene configurato il tempo per la chiusura automatica tramite il segnale *B3*, incrementandolo di soli 10 secondi in quanto la logica è simile al caso precedente. Si richiede quindi l'apertura del cancello entro 10 secondi, durante i quali il *led giallo* lampeggia. Una volta che il cancello è aperto, tutti i led sono accesi. Trascorso il tempo necessario per la chiusura automatica (20 secondi, ovvero 10 del valore iniziale del timer più 10 dell'incremento effettuato), il cancello si chiude automaticamente in 10 secondi, durante i quali il *led giallo* lampeggia. Infine, il cancello è correttamente chiuso quando *P2* diventa *true* e tutti i led sono spenti.

> Nota: Nella traccia viene indicato B2 come pulsante per il la chiusura automatica e B3 come pulsante del tempo di lavoro. In questo progetto sono stati invertiti.

Condizione_errore

Per creare il test **condizione_errore** si suppone che il cancello sia in fase di chiusura, con il led giallo lampeggiante. Tuttavia, al termine del tempo di lavoro, il cancello non risulta chiuso (*P2=false*). Dopo 10 secondi, il segnale *P2* è ancora false, indicando che il cancello non ha completato la chiusura. A questo punto, il sistema segnala un errore accendendo il led rosso. Il led rosso rimane acceso finché il sistema non viene riavviato.

3.3 Test Cancello Ostacoli

ostacolo_cancello_chiuso

Il test **ostacolo_cancello_chiuso** inizia con il cancello chiuso e tutti i led spenti. Quando viene segnalata la presenza di un ostacolo tramite il segnale *P1*, i led rimangono spenti. Successivamente,

quando viene richiesta l'apertura del cancello tramite il segnale *B1*, la richiesta viene ignorata a causa dell'ostacolo. In risposta, il LED verde inizia a lampeggiare a una frequenza di 1 Hz per 30 secondi. Dopodiché tutti i led si spengono e il cancello rimane chiuso.

ostacolo_cancello_in_movimento

Il test **ostacolo_cancello_in_movimento** inizia con il cancello chiuso e i led spenti. Si richiede l'apertura del cancello tramite il segnale *B1*, e il led giallo lampeggia per 10 secondi a una frequenza di 0.5 Hz. Quando il cancello è completamente aperto, tutti i led sono accesi. Successivamente, viene richiesta la chiusura del cancello ancora tramite il segnale *B1*. Viene segnalata la presenza di un ostacolo con il segnale *P3*, dopo 6 secondi dall'inizio della chiusura, il quale provoca l'immediata riapertura del cancello, il led giallo lampeggia a una frequenza di 0.5 Hz. Il test si conclude con il cancello aperto e tutti i led accesi.

Codice sorgente e dimostratore hardware

In questo capitolo, verrà illustrato il modello Stateflow adeguatamente adattato per permettere la generazione e il caricamento del firmware sulla scheda STM32 Nucleo G474RE, insieme alla configurazione necessaria. Inoltre, verranno descritti i test condotti direttamente sull'hardware per verificare le funzionalità implementate.

4.1 Realizzazione del circuito

Come da specifiche, i componenti utilizzati per la realizzazione del circuito sono i seguenti:

- *Pulsante di apertura* (B1)
- *Pulsante per la chiusura automatica* (B2)
- *Pulsante del tempo di lavoro* (B3)
- *Sensore di presenza ostacoli* (P1)
- *Sensore di chiusura* (P2)
- *Led di errore* (L1)
- *Led di movimento* (L2)
- *Led di presenza ostacoli* (L3)

In particolare, la configurazione utilizzata è indicata nella seguente tabella:

Componente	**pin**
B1	PC_8
B2	PC_10
B3	PC_13
P1	PC_6
P2	PC_5
L1	PB_5
L2	PB_4
L3	PB_3

Pertanto, il circuito realizzato è rappresentato nella seguente immagine:

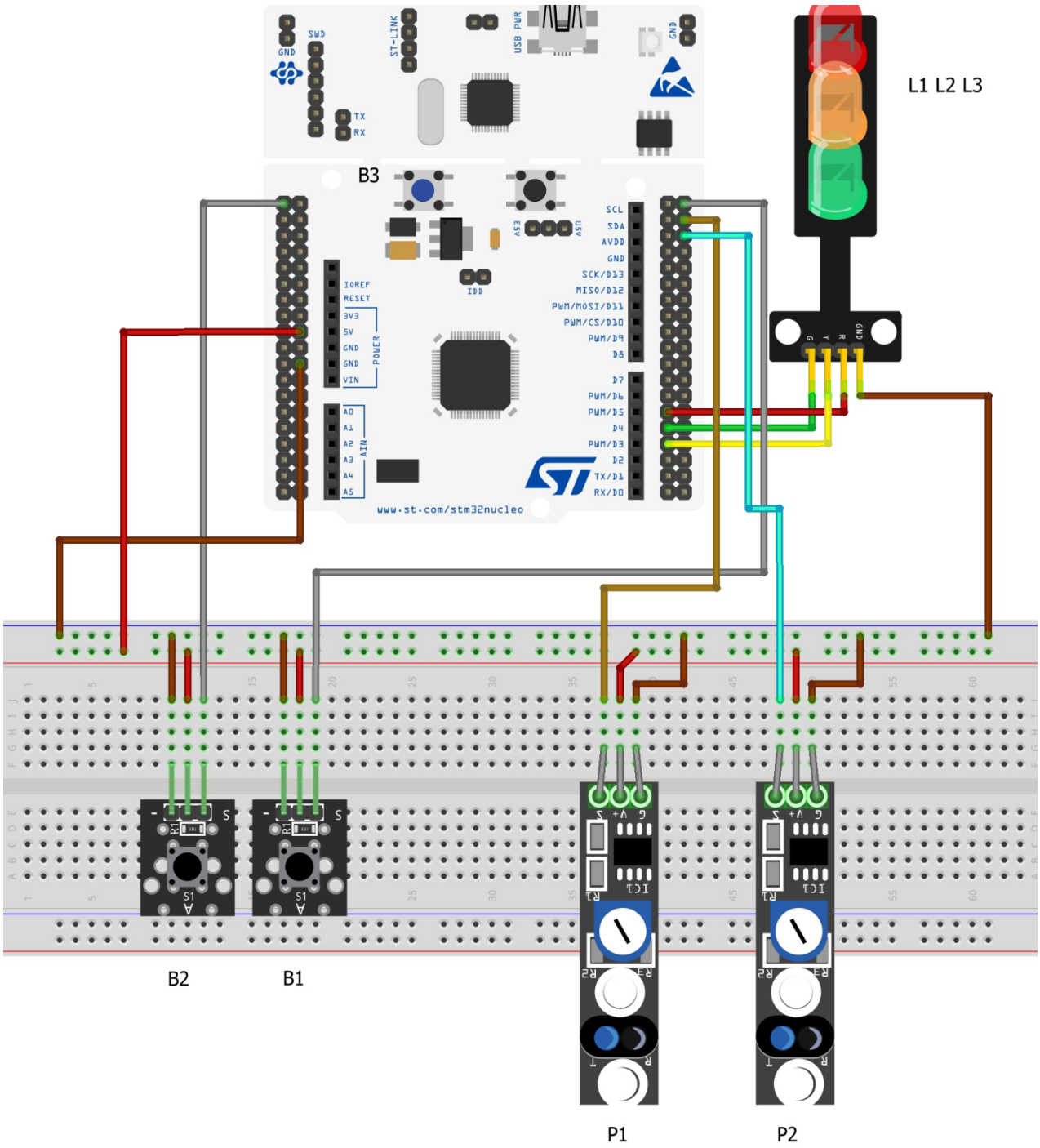

Figura 22: Circuito realizzato con Fritzing

4.2 Generazione del codice sorgente

Per procedere con la generazione del codice sorgente e successivo caricamento sulla scheda, è stato necessario adattare lo **statechart** precedentemente realizzato.

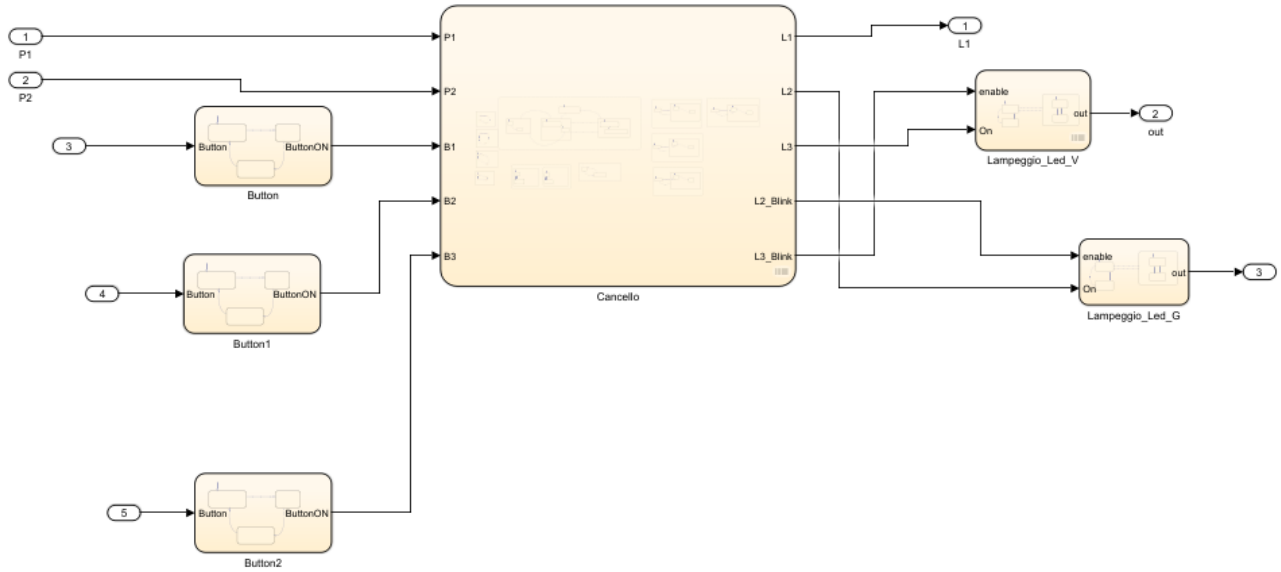

Figura 23: Statechart adattato

A questo punto, attraverso l'utilizzo del plugin *Embedded Coder*, dopo aver configurato il modello della scheda per cui effettuare la *build*, si è proceduto con il *deploy* del firmware. Inoltre, al fine di verificare che tutto fosse andato a buon fine, sono stati fisicamente eseguiti tutti i diagrammi presentati nel Capitolo 1 e non, per i quali è stato riscontrato un esito positivo.

Indice delle Figure

FIGURA 1: ACCENSIONE DEL DISPOSITIVO 5
FIGURA 2: CONFIGURAZIONE DELLE IMPOSTAZIONI DEL CANCELLO 6
FIGURA 3: FUNZIONAMENTO 7
FIGURA 4: PRESENZA OSTACOLI 8
FIGURA 5: VERIFICA CHIUSURA 8
FIGURA 6: USE CASE DIAGRAM GENERALE 9
FIGURA 7: APERTURA/CHIUSURA 10
FIGURA 8: RILEVAMENTO OSTACOLI 11
FIGURA 9: CONFIGURAZIONE DEI TEMPI DI OPERAZIONE DEL CANCELLO 12
FIGURA 10: ACCENSIONE DEL SISTEMA E VERIFICA SENSORE CHIUSURA 13
FIGURA 11: VERIFICA CORRETTA CHIUSURA DEL CANCELLO 14
FIGURA 12: STATE DIAGRAM 16
FIGURA 13:STATEFLOW CANCELLO AUTOMATICO 18
FIGURA 14:BUTTON CHART 19
FIGURA 15: LAMPEGGIO LED CHART 20
FIGURA 16: CANCELLO CHART 24
FIGURA 17: FUNCTION 24
FIGURA 18: UPDATE 25
FIGURA 19: OSTACOLO 25
FIGURA 20: TIMER_T_C 26
FIGURA 21: CANCELLO 26
FIGURA 22: CIRCUITO REALIZZATO CON FRITZING 31
FIGURA 23: STATECHART ADATTATO 32